SpecField: Checking Field-Level Conformance of Protocol Implementations to Standards

***Abstract*—Field requirements in protocol standards are usually described in natural language, while the corresponding implementation logic is scattered across parsing, validation, and error-handling code, making it difficult to directly connect standard requirements with implementation behavior. This paper presents SpecField, a framework for field-level protocol conformance checking. The core idea is to use protocol fields as semantic anchors between natural-language standards and source-code implementations: SpecField extracts structured field rules from standards, aligns them with variables and code contexts in implementations, and uses traceable evidence to judge whether implementation behavior deviates from standard requirements. For suspected deviations, SpecField further generates maintainer-oriented defect reports and reproducible validation artifacts to support manual confirmation and result auditing. We submitted 204 reports on real protocol implementations including TLS/DTLS and QUIC; 126 have been confirmed or fixed, 9 were rejected as false positives, and 69 remain pending. Two confirmed issues were assigned previously unknown CVEs.**

***Index Terms*—Protocol analysis, conformance checking, protocol implementation, large language models, specification inference, software testing.**

I. Introduction

Network protocol standards are a key foundation for interoperability and security across independent implementations. Protocols such as TLS [1], [2], MQTT [3], [4], and QUIC [5]–[7] specify not only how endpoints parse messages, but also how they negotiate parameters, update state, authenticate peers, and handle exceptional inputs. A deviation from these requirements does not necessarily become an exploitable vulnerability immediately, but it can still introduce practical risk. For example, an implementation may accept a field value forbidden by the standard, omit a state-dependent check, incorrectly reject a valid message, or trigger abnormal error-handling behavior under a specific configuration. Such deviations may cause interoperability failures and may further enable downgrade attacks, denial-of-service conditions, or other security risks.

Detecting these deviations is difficult because standards and code usually express the same behavior in different forms. Protocol standards are natural-language documents whose requirements are distributed across tables, field descriptions, state-machine descriptions, compatibility notes, and exception clauses. RFC-style keywords can indicate requirement strength [8], [9], and prior work has used RFC-derived rules, parser specifications, and large-language-model agents to analyze protocol implementations [10]–[13]. However, these approaches usually analyze rules, interfaces, or protocol behaviors as their main objects, and their granularity is relatively coarse. They therefore struggle to precisely capture the correspondence between individual fields, their related conditions, and the implementation behavior that realizes them.

This paper focuses on field-level protocol requirements. Many actionable requirements specify that a field may appear only in a particular state, that its value must fall within an allowed range, that it must match another field, that it depends on negotiated parameters, or that it must trigger rejection under an invalid combination. We represent such requirements as $\langle C, f, A\rangle$: under condition C, field f must satisfy action or constraint A. This form is not a complete formal semantics, but it provides a clear boundary for checking standard text against implementation code.

We present SpecField, a conformance-checking framework for natural-language protocol standards and source-code implementations. SpecField first extracts the above rules, then retrieves syntax-aware code contexts related to field parsing, value validation, state dependencies, and error handling, and finally uses iterative reasoning to determine whether the implementation satisfies the requirement. Each result is reported as CONSISTENT, INCONSISTENT, or INSUFFICIENT, with supporting or missing evidence recorded explicitly. For suspicious inconsistent results, SpecField further attempts to generate executable validation tests.

We evaluate SpecField on real-world implementations of TLS/DTLS, MQTT, QUIC, and CoAP. SpecField produced 204 submitted conformance-deviation reports; maintainers confirmed or fixed 126, rejected 9 as false positives, and 69 remain pending. Two confirmed reports were assigned previously unknown CVEs. Under GPT-5.4 pricing, the average model cost is $1.122 per submitted report.

Our main contributions are:

- We formulate protocol conformance checking as a field-centric alignment problem between standards and implementations, and design SpecField. The framework extracts structured field rules from natural-language standards and aligns them with implementation code contexts to identify conformance deviations in implementations.
- We introduce an auditable reporting workflow that turns model-assisted analysis into maintainer-oriented defect reports and reproducible validation artifacts, reducing reliance on raw LLM verdicts and supporting manual confirmation.
- We evaluate SpecField on real-world implementations of TLS/DTLS, MQTT, QUIC, and CoAP, producing 204 submitted reports, 126 maintainer-confirmed or fixed reports, 9 rejected false positives, 69 pending reports, and two previously unknown CVEs.

II. Background

A. Field-Level Conformance

Protocol conformance asks whether an implementation satisfies the behavioral requirements stated by a standard. In natural-language standards, many requirements are attached to protocol fields: message fields, parameters, lengths, flags, and valid field combinations. We call this scope *field-level semantic conformance*.

Field-level conformance requires two links: mapping the standard's field and condition to implementation objects, and checking whether the relevant code path enforces the required behavior. This scope is narrower than full protocol verification, but it covers many practical deviations because parsing, validation, negotiation, and error handling are often organized around fields.

B. Normative Modalities

Normative modality determines whether a statement is treated as a reportable obligation. SpecField extracts *MUST* and *MUST NOT* requirements as primary conformance rules, tracks *SHOULD* requirements as lower-severity deviations, and ignores *MAY* statements because they describe optional behavior. The modality is stored with each rule and used when ranking reports.

III. Method

Figure 1 shows the workflow of SpecField. Given a protocol standard S and an implementation P, SpecField checks whether P satisfies the normative field-level requirements stated in S. The system first builds a field space from the standard, extracts structured rules over those fields, maps each rule to implementation variables and code contexts, and finally uses an evidence-bounded reasoning-and-test agent to decide whether the observed implementation behavior is consistent with the requirement. The output is not only a label, but also the standard evidence, code evidence, uncertainty notes, validation results when available, and reproduction artifacts for maintainer triage.

A. Problem Definition

SpecField represents each standard-derived requirement as a rule $r = \langle f, C, A, M, E \rangle$. Here, f is the constrained protocol field, C is the condition under which the rule applies, A is the required action or constraint, M is the normative modality such as *MUST* or *SHOULD*, and E records source evidence from the standard. For each rule, the checker returns $o = \langle y, E, U, T, R \rangle$, where $y \in \{\textsc{Consistent}, \textsc{Inconsistent}, \textsc{Insufficient}\}$ is the judgment, E contains both document and code evidence, U records missing or uncertain context, T stores validation-test evidence when the validation stage runs, and R is an optional inconsistency reason used only when $y = \textsc{Inconsistent}$.

This representation separates standard-derived obligations from implementation judgments. A rule records what the standard requires, under which condition it applies, and where the requirement appears, while a judgment records whether the corresponding implementation evidence satisfies that rule. By making the field, condition, action, and source evidence explicit, SpecField aligns standard requirements with implementation variables, code contexts, and runtime behavior using both semantic and syntactic evidence.

B. Field-Space Construction

Directly extracting requirements from open-ended prose is unstable because standards mix normative rules with design motivation, compatibility advice, examples, and implementation guidance. SpecField therefore starts by constructing a *field space*: a set of protocol objects that later rules may reference. Field candidates are collected from message formats, extension structures, parameter tables, and field-definition paragraphs. For each field, SpecField records its canonical name, aliases, parent message or structure, type, allowed values when available, and source locations.

SpecField first preprocesses input protocol standards, including RFC-style specifications and other protocol standards provided in formats such as PDF, DOC, or TXT. To preserve local semantic coherence, it splits each document into paragraphs and greedily merges consecutive paragraphs into bounded text blocks without splitting a paragraph in the middle. When adding the next paragraph would exceed the configured threshold, the current block is committed and a new block starts from that paragraph. The default threshold is 6000 characters and can be adjusted by configuration. These stable, tokenizer-independent blocks are used for both field discovery and later rule extraction.

The field space defines a constrained set of protocol objects that can be referenced during rule extraction. Since protocol standards frequently reuse generic field names across different message structures, field references cannot be reliably resolved without local structural context. For example, names such as `length`, `type`, and `flags` may denote different fields in different messages, while descriptive terms such as "compatibility mode" may refer to protocol behavior rather than concrete field objects. Therefore, SpecField resolves each field reference by jointly considering its parent message or structure, aliases, semantic role, and source location. Two field references are merged only when these contextual attributes are compatible; otherwise, the reference is retained as ambiguous or excluded from the scope of field-level checking. This conservative resolution strategy reduces erroneous field merges and prevents such errors from propagating to subsequent rule-to-code alignment and consistency reasoning.

C. Field-Centric Rule Extraction

After constructing the field space, SpecField extracts field-centric rules from normative standard sections. For each section, the model receives the section text, the field-space entries relevant to that section, and a lexicon of RFC 2119-style normative keywords. Within this constrained context, it identifies normative statements, resolves mentioned protocol objects to field-space entries, and emits normalized field-level rules.

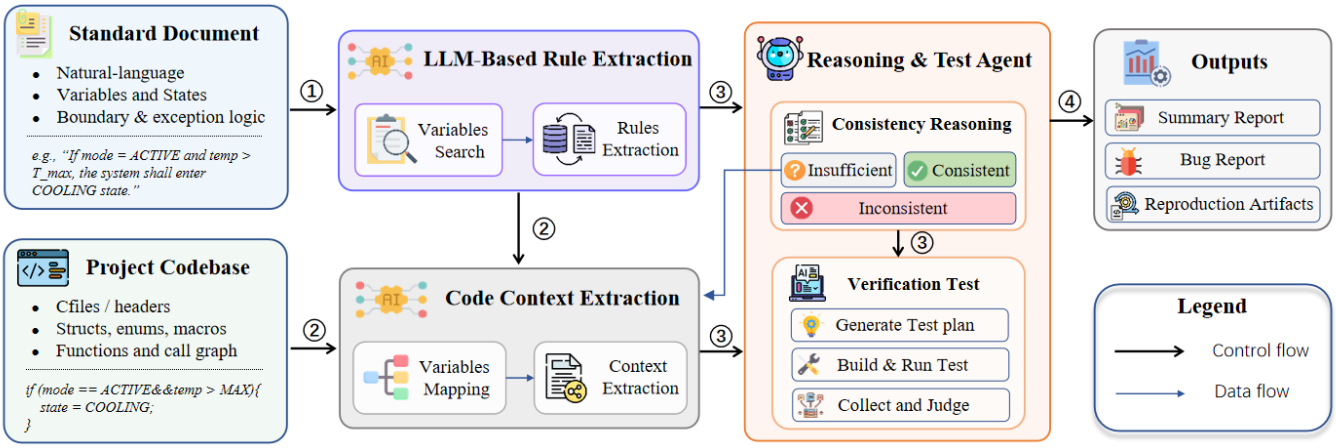

Fig. 1. Overview of SpecField. The system extracts field-level rules from a standard, aligns them with implementation evidence, and uses bounded reasoning and validation tests to produce reports and reproduction artifacts.

For example, RFC 8446 specifies the TLS 1.3 `Cookie` extension as a `Cookie` structure whose `cookie` field is encoded as `opaque cookie<1..2^16-1>` [2]. Since this vector definition permits only non-empty values, a `HelloRetryRequest` carrying a zero-length `Cookie` violates the field bound. SpecField normalizes this standard constraint into the following simplified field-level rule:

```
{
  "f": "Cookie.cookie",
  "C": "Client receives a TLS 1.3 HRR Cookie.",
  "A": "Reject if Cookie is empty.",
  "M": "MUST",
  "E": "RFC8446 Sec.4.2.2: ...cookie<1..2^16-1>..."
}
```

SpecField confines rule extraction to local sections and a known field set, rather than asking the model to infer rules from the whole standard at once. The field space provides a traceable referential basis for protocol objects, so each emitted rule can be linked back to concrete fields and source text. For normative sentences that contain multiple constraints, SpecField splits them into multiple field-level rules, each corresponding to an independent checkable behavior such as field presence, value constraints, range checks, state updates, or rejection behavior.

The extracted rules are further normalized into action families that are close to implementation behavior, including *assign*, *derive*, *select*, *require-present*, *require-absent*, *compare*, *range-check*, *state-update*, and *reject*. When one sentence contains multiple constraints, SpecField atomizes it into multiple rules so each rule can be aligned and tested independently. Negated requirements are rewritten into explicit actions; for example, "MUST NOT accept x" becomes an expected *reject* action under condition x. Cross-references are expanded when the referenced section defines a relevant field or bound, while the normalized rule remains linked to the original text.

A deterministic checker validates the model output before alignment. It checks that field references resolve to field-space entries, the cited source span exists, the action type belongs to a supported action class, and the candidate does not duplicate an existing record. Invalid outputs are retried with a concise repair prompt; candidates that still fail are discarded rather than turned into unauditable rules. Normative text outside the field-level scope, such as performance guidance or cryptographic assumptions requiring proof, is recorded as out of scope instead of silently lost.

D. Syntax-Aware Rule-to-Code Alignment

The alignment stage maps each rule r to implementation variables and code contexts. It combines semantic ranking with syntax-aware evidence because pure semantic retrieval can select comments that resemble the standard but do not manipulate the constrained field, while pure exact matching misses renamed or split representations.

File ranking and variable localization. Given a field-level rule, SpecField first narrows the search space to files that are likely to contain the field representation or its checking logic. It summarizes candidate C/C++ files using lightweight structural and lexical signals, such as file paths, declarations, comments, and field-alias hits. An LLM-based ranker then selects the most relevant files based on the rule, action class, aliases, and file summaries. Within the ranked files, SpecField localizes candidate implementation variables for the constrained field. It constructs aliases from the field name, its parent message or structure, and common naming variants, and builds a Tree-sitter-based symbol index [14] over program entities such as variables, structure members, macros, and parameters. Candidate identifiers are ranked by how well they match the field aliases and by whether their usage is relevant to the rule action. For example, identifiers used in bounds checks, comparisons, state updates, or error-return paths are prioritized over generic temporaries or identifiers without action-relevant usage.

Context retrieval. For each candidate variable v, SpecField retrieves snippets from exact textual hits, enclosing symbol

scopes, and semantic windows whose tokens overlap the rule. Before LLM filtering, each snippet c receives the score:

$$S_{\text{ctx}}(c) = w_{\text{exact}} \cdot \mathbb{I}_{\text{exact}}(c) + w_{\text{var}} \cdot |\mathcal{T}_c \cap \mathcal{T}_v| + w_{\text{rule}} \cdot |\mathcal{T}_c \cap \mathcal{T}_r| + b(c). \quad (1)$$

Here, $\mathbb{I}_{\text{exact}}(c)$ is an indicator for exact evidence: it is 1 when snippet c contains the candidate variable or one of its aliases, and 0 otherwise. $\mathcal{T}_c$, $\mathcal{T}_v$, and $\mathcal{T}_r$ denote the token sets of the code snippet, the candidate variable, and the rule, respectively. The bonus term $b(c)$ rewards action-relevant usage, such as initialization, validation, comparison, rejection, error propagation, configuration reads, and state updates. We fix the weights to $w_{\text{exact}} = 5$, $w_{\text{var}} = 0.9$, and $w_{\text{rule}} = 0.28$. These constants encode an ordinal retrieval preference: direct field evidence should dominate variable-token overlap, which should dominate weak rule-token overlap. Snippets without exact variable or alias occurrence are capped to prevent broad semantic similarity from overwhelming direct implementation evidence.

The top snippets are then sent to an LLM semantic filter. The filter sees only the rule and ranked snippets, selects the contexts most relevant to the rule, and explains the correspondence between each selected snippet and the expected behavior. The result is an evidence bundle containing field representations, checking logic, call relationships, configuration conditions, and error paths for the next stage.

For the empty-Cookie rule, SpecField localizes the client-side parsing logic related to the `cookie` field during context retrieval and retrieves the following potentially relevant OpenSSL code snippet. The snippet shows how the OpenSSL client reads the Cookie extension, stores its content and length, and reaches the `decode_error` path when length parsing fails. During alignment, this snippet is added to the evidence bundle as candidate code evidence; whether a zero-length Cookie check is present is further determined in the subsequent reasoning and validation stage.

```
if (!PACKET_as_length_prefixed_2(pkt, &cookie)
    || !PACKET_memdup(&cookie, &s->ext.tls13_cookie,
        &s->ext.tls13_cookie_len)) {
    SSLfatal(s, SSL_AD_DECODE_ERROR,
                    SSL_R_LENGTH_MISMATCH);
    return 0;
}
```

E. Consistency Reasoning and Validation

After rule-to-code alignment, the reasoning-and-test agent receives an evidence bundle containing the field rule, standard span, localized variables, selected code snippets, call relationships, and configuration conditions. It first performs static consistency reasoning over this bounded bundle. If the evidence implements the required check or handling, the label is CONSISTENT; if it contradicts the rule, the label is INCONSISTENT. If the current evidence lacks key code context needed to settle the rule, such as macro definitions, configuration branches, validation helpers, or error-propagation paths, the available evidence is insufficient to support a determinate consistency judgment, and the label is INSUFFICIENT. In this case, U records the missing information to be retrieved and stores the targeted retrieval request generated by the LLM for the next round.

Algorithm 1: Evidence-driven consistency reasoning.

Input: Rule r, implementation index I, maximum rounds K
Output: Consistency tuple $\langle y, E, U, T, R \rangle$

$C_0 \leftarrow \text{AlignRuleToCode}(r, I)$
for $i \leftarrow 0$ **to** K **do**
  $o_i \leftarrow \text{Reason}(r, C_i)$
  if $o_i.y \neq \text{Insufficient}$ **then**
    return o_i
  $q_i \leftarrow \text{ContextRequest}(o_i.U)$
  $C_{i+1} \leftarrow C_i \cup \text{RetrieveAdditionalContext}(q_i, I)$
  if $C_{i+1} = C_i$ **then**
    return o_i
return o_K

Algorithm 1 presents the evidence-completion loop used in consistency reasoning. Starting from the initial evidence bundle C_0 produced by rule-to-code alignment, SpecField invokes the reasoning step to obtain a judgment o_i. If the judgment is CONSISTENT or INCONSISTENT, the algorithm returns the result directly. If the judgment is INSUFFICIENT, the missing information recorded in U is converted into a targeted retrieval query, and the newly retrieved context is merged into the evidence bundle for the next round. The loop stops when a determinate judgment is reached, no new context can be retrieved, or the maximum number of rounds is reached. This design allows SpecField to postpone a decision when the current evidence is incomplete, while continuing to expand the code context needed to settle the rule.

Static reasoning can only judge the retrieved code evidence, so it may mistake an infeasible branch, code guarded by build options, or a path already filtered by an upper-level check for actual implementation behavior. To reduce such errors, SpecField further validates candidate inconsistencies with generated tests. The validation-test agent constructs inputs from the field rule and defect hypothesis, and drives the implementation through a unit test, parser harness, or command-line execution to reach the relevant code path. A candidate is kept as a reproducible deviation only when the execution shows that the implementation accepts an input that violates the standard, or rejects an input that satisfies it; otherwise, the static hypothesis is not promoted to a confirmed report.

The OpenSSL empty-Cookie case illustrates this process. SpecField first extracts a field rule from RFC 8446: a TLS 1.3 client should reject a zero-length `Cookie`. During static reasoning, SpecField examines the aligned parser context and finds that the OpenSSL client reads and stores the Cookie data and its length through `PACKET_as_length_prefixed_2` and `PACKET_memdup`, but this context contains no explicit zero-length check. SpecField therefore treats it as a candidate inconsistency and passes it to the validation test for further confirmation. The validation stage constructs a

HelloRetryRequest carrying an empty Cookie extension and observes that the client does not abort the handshake immediately, but instead sends a second ClientHello before failing later [15]. The final inconsistency record contains the field rule, code evidence, runtime observation, and inconsistency explanation:

```
{
  "y": "Inconsistent",
  "E": {
    "document": {
      "section": "RFC8446 Sec.4.2.2",
      "text": "opaque cookie<1..2^16-1>"
    },
    "code": {
      "file": "ssl/statem/extensions_clnt.c",
      "lines": "2372-2377",
      "text": "if (!PACKET_as_length_prefixed_2..."
    }
  },
  "U": [],
  "T": {
    "input": "HRR with an empty Cookie extension",
    "oracle": "Client should abort immediately",
    "observed": "Client sends a second ClientHello
     before later failure"
  },
  "R": "When a TLS 1.3 HelloRetryRequest carries an
     empty Cookie, OpenSSL does not strictly enforce
     RFC 8446. According to the RFC,..."
}
```

For confirmed inconsistencies, SpecField generates structured reports and reproduction artifacts, recording the field rule, related standard text, key code snippets, inconsistency rationale, test input, build configuration, execution command, observed result, and log summary. This forms a traceable evidence chain from the standard requirement to code evidence and then to runtime behavior, supporting later review and reproduction. For unconfirmed results, the system does not promote the static defect hypothesis to a confirmed issue; instead, it records the blocking condition, such as a build failure, an unreachable target path, a missing test entry point, or an insufficient observation signal.

IV. Evaluation

We evaluate SpecField on real protocol implementations and answer five research questions:

- **RQ1: Field-centric rule discovery.** How effectively does SpecField discover field-level rules from protocol standards?
- **RQ2: Rule-to-code alignment.** How accurately does SpecField map a standard rule to implementation variables and contexts?
- **RQ3: Bug finding.** How many conformance-deviation reports does SpecField discover, and how many are confirmed, fixed, rejected, pending, or assigned CVEs? How do different models perform?
- **RQ4: Cost.** What is the approximate model-calling cost per submitted report when SpecField uses different models?
- **RQ5: Ablation.** Which system components are responsible for the observed performance?

A. Subjects and Baselines

Table I lists the evaluation subjects used in this study. To ensure a fair comparison across different implementations, we require *all implementations of the same protocol version to share the same rule set* extracted from the corresponding standard. For example, all TLS 1.3 implementations are checked against the same 312 rules extracted from RFC 8446 [2], and the two QUIC implementations are checked against the same 286 rules extracted from RFC 9000 [5]. Specifically, the TLS/DTLS subjects include OpenSSL [16], wolfSSL [17], Mbed TLS [18], openHiTLS, and BoringSSL [19]; the QUIC subjects include quiche [20] and ngtcp2 [21]; the MQTT subjects include Mosquitto [22] and wolfMQTT [23]; and the CoAP subject is libcoap [24].

TABLE I
Protocol implementations used for evaluation.

Subject	Version	LoC	Protocol	Specification
OpenSSL	11b7b6e	1456.3K	TLS 1.3	RFC 8446
wolfSSL	1d363f3	520.4K	TLS 1.3	RFC 8446
Mbed TLS	0fe989b	310.8K	TLS 1.3	RFC 8446
openHiTLS	3d295814	480.2K	TLS 1.3	RFC 8446
Mosquitto	99fa50f	180.6K	MQTT 5.0	MQTT 5.0
Mosquitto	99fa50f	180.6K	MQTT 3.1.1	MQTT 3.1.1
wolfMQTT	88d37ed	41.5K	MQTT 3.1.1	MQTT 3.1.1
quiche	851533c	92.3K	QUIC	RFC 9000
ngtcp2	4af1c77	130.1K	QUIC	RFC 9000
libcoap	7bc9a41	115.0K	CoAP	RFC 7252
BoringSSL	7f4d9a2	760.4K	DTLS 1.3	RFC 9147

We compare the full SpecField pipeline with simpler configurations while keeping the subject versions, rule budgets, model family, temperature setting, and maximum reasoning rounds fixed. **Direct Agent** uses the same standard chunking as SpecField, but asks an agent to compare the standard text directly with the implementation code and produce conformance reports, without constructing a field space or field-level rules. **Generic RAG** keeps the same report-generation task but replaces SpecField's field-aware context retrieval with generic embedding-based retrieval over code chunks. **Field Rules** keeps SpecField's field-rule representation and code indexes, but performs only a single reasoning pass without iterative evidence completion. The remaining configurations add iterative reasoning, validation testing, or a forced-binary diagnostic control. Each configuration uses the same top-15 file, top-8 variable, top-12 snippet, and up to three evidence-completion rounds unless that component is absent by design; prompt templates and raw outputs are included in the artifact.

B. Metrics and Ground Truth

For rule extraction, we run each extraction configuration three times to reduce missed rules caused by stochastic LLM decoding. Candidate rules from the three runs are first pooled and deduplicated by protocol version, source span, constrained field, applicability condition, expected action, and modality, forming the candidate pool for that configuration.

We then compare the pool against the manually annotated benchmark of field-level obligations. This benchmark was annotated by researchers from the TLS 1.3 and MQTT 3.1.1

standards and contains 430 field-level obligations; each obligation includes the source span, constrained field, condition, expected behavior, and modality. If a candidate rule, after normalization, matches a gold obligation in field, condition, action, and source semantics, it is counted as a true positive (TP). If a candidate rule matches no gold obligation, or corresponds to unsupported normative text or erroneous splitting, it is counted as a false positive (FP). If a gold obligation is not discovered in any of the three runs for that configuration, it is counted as a false negative (FN). The precision, coverage, and F1 values in Table II are computed by matching the pooled candidate rules against the 430 gold obligations.

To evaluate code-context extraction, we use Var@1, Var@3, Ctx@1, Ctx@3, MRR, and average token consumption. We stratify-sample 312 rule-implementation pairs by protocol and implementation. Two researchers with protocol and systems experience independently annotate the correct implementation variable in each sample and the minimal code context needed to determine the rule's behavior. Disagreements are resolved by joint review of the standard span, callers, validation helpers, and tests.

Here, Var@1 indicates whether the top-ranked candidate implementation variable is the gold variable; Var@3 indicates whether the gold variable appears among the top three candidates. Ctx@1 indicates whether the top-ranked code context contains the key evidence required to judge the rule, such as a field representation, a relevant check, error propagation, or an error-handling path; Ctx@3 indicates whether at least one of the top three code contexts contains such evidence. MRR measures the average rank of the first correct implementation variable in the ranked candidate-variable list. A correct variable ranked first contributes 1, one ranked second contributes 1/2, one ranked third contributes 1/3, and a sample with no correct variable in the candidate list contributes 0. Average token consumption measures the number of LLM tokens consumed per rule-implementation pair during code-context extraction.

To evaluate the vulnerability-finding capability of different models, we run the full pipeline with four model families, namely GPT, Claude, DeepSeek, and Qwen, and repeat the experiment three times independently for each model. Candidate reports from the twelve runs are first pooled and deduplicated by protocol version, implementation, normative rule, and observed behavioral deviation, so that the same issue is not counted multiple times. Security experts then manually adjudicate each deduplicated report according to predefined correctness criteria; the reports judged as true conformance deviations form the shared true-report set.

For a given model, a true report is counted as discovered if the model finds it in at least one of its three runs. We define pooled coverage as the fraction of reports in the shared true-report set found by that model. This metric measures discovery over the shared true-report set and is not a claim of whole-corpus recall.

TABLE II
FIELD-RULE DISCOVERY PERFORMANCE.

Method	Extract.	TP	FP	FN	Prec.	Cov.	F1
0-shot prompt	319	263	56	167	0.82	0.61	0.70
2-shot prompt	384	323	61	107	0.84	0.75	0.79
Schema-only	402	347	55	83	0.86	0.81	0.83
SpecField	439	407	32	23	0.93	0.95	0.94

C. RQ1: Field-Centric Rule Discovery

Table II evaluates field-rule discovery on the manually annotated TLS 1.3 and MQTT 3.1.1 benchmark. For each extraction configuration, we independently run the extractor three times, then pool and deduplicate the candidate rules from the three runs before matching them against the 430 gold field-level obligations. We compare against three baselines: 0-shot prompt directly extracts rules from the standard text; 2-shot prompt adds a few examples to guide the model output; and Schema-only uses the same structured output schema and rule validators as SpecField, but does not construct the field space used to constrain and disambiguate field references. All methods use the same standard chunking and model settings.

SpecField achieves the highest precision, coverage, and F1. The jointly higher coverage and precision indicate that SpecField recovers more manually annotated field-level obligations, rather than simply expanding the candidate set at the cost of precision. The field space provides clear protocol-object boundaries for rule extraction: it reduces false rules from descriptive text, compatibility notes, or generic concepts, while helping the model resolve aliases, contextual references, and cross-paragraph field constraints.

D. RQ2: Rule-to-Code Alignment

Table III shows that code retrieval based only on lexical or semantic similarity has limited effectiveness. Exact Match relies on lexical overlap, BM25 uses sparse lexical retrieval, and Embedding uses dense semantic retrieval. In contrast, LLM Direct feeds retrieved code directly to the LLM and substantially outperforms these non-LLM methods in both variable recovery and context recovery, showing that LLM semantic judgment helps identify relevant code evidence. SpecField further uses field-level rules to guide code localization, narrowing the candidate code range before LLM filtering and thereby reducing interference from irrelevant code. As a result, it achieves the best overall performance in both variable and context recovery.

SpecField achieves 0.79 Var@1 and 0.91 Var@3, showing that correct implementation variables are usually ranked near the top. It also achieves 0.74 Ctx@1 and 0.87 Ctx@3, indicating that the retrieved contexts often contain decisive evidence for rule checking, such as validation logic, error propagation, or state updates. This improvement is important because many false positives arise when retrieval returns nearby but semantically irrelevant functions. Meanwhile, SpecField consumes 5.2K tokens on average, compared with 7.1K tokens for LLM Direct. This is because field-guided localization narrows

TABLE III
EFFECTIVENESS OF RULE-TO-CODE ALIGNMENT.

Method	Var@1	Var@3	Ctx@1	Ctx@3	MRR	Tok.
Exact Match	0.34	0.48	0.29	0.43	0.41	0.9K
BM25	0.41	0.56	0.36	0.51	0.48	1.3K
Embedding	0.47	0.64	0.43	0.59	0.55	2.5K
LLM Direct	0.72	0.84	0.69	0.88	0.78	7.1K
SpecField	0.79	0.91	0.74	0.87	0.84	5.2K

TABLE IV
SENSITIVITY OF RULE-TO-CODE ALIGNMENT SIGNALS.

Setting	w_{exact}	w_{var}	w_{rule}	Var@1	Ctx@1	MRR
Default	5.0	0.90	0.28	0.79	0.74	0.84
No exact	0.0	0.90	0.28	0.70	0.63	0.75
No var. tok.	5.0	0.00	0.28	0.73	0.66	0.78
No rule tok.	5.0	0.90	0.00	0.77	0.72	0.82
No action	5.0	0.90	0.28	0.75	0.69	0.80
$w_{\text{exact}} \times 0.5$	2.5	0.90	0.28	0.78	0.73	0.83
$w_{\text{exact}} \times 2$	10.0	0.90	0.28	0.79	0.74	0.84
$w_{\text{var}} \times 0.5$	5.0	0.45	0.28	0.78	0.73	0.83
$w_{\text{var}} \times 2$	5.0	1.80	0.28	0.79	0.74	0.84
$w_{\text{rule}} \times 0.5$	5.0	0.90	0.14	0.78	0.73	0.83
$w_{\text{rule}} \times 2$	5.0	0.90	0.56	0.78	0.73	0.83
Uniform weights	1.0	1.00	1.00	0.74	0.68	0.79

the candidate code range before LLM filtering, so the LLM performs a more focused relevance judgment over fewer and more relevant contexts.

a) Context-retrieval prior ablation.: The coefficients in Equation (1) are fixed retrieval priors rather than protocol-specific learned weights. Table IV ablates the retrieval signals and perturbs their weights; "var. tok." and "rule tok." abbreviate variable-token and rule-token overlap. The results show that removing exact variable matches causes the largest performance drop, while removing weaker terms leads to more gradual degradation in alignment quality. When the exact-variable, variable-token, and rule-token weights are scaled within a factor of two while preserving their relative order, the alignment metrics change only modestly in these tested settings. In contrast, the largest degradation appears when the retrieval-signal hierarchy is broken, especially when exact variable matches are removed, $b(c)$ is disabled, or all weights are assigned uniform values. These results suggest that SpecField's alignment quality mainly comes from the retrieval-signal hierarchy and syntax-aware features, rather than from a narrowly tuned set of constants.

E. RQ3: Bug Finding Results

Table V summarizes the status of reports submitted to repository maintainers. It reports the final deduplicated submissions and their maintainer outcomes.

Sub. denotes submitted reports: deduplicated candidate issues submitted to maintainers with a standard text span, a normalized rule, code evidence, and reproduction material when available. **Conf.** denotes confirmed reports that were fixed by maintainers or explicitly acknowledged as valid. **FP** denotes false positives rejected by maintainer feedback or expert adjudication. **Pend.** denotes pending reports without sufficient external signal for final adjudication.

TABLE V
STATUS OF SUBMITTED CONFORMANCE-DEVIATION REPORTS AND MAINTAINER OUTCOMES. #R DENOTES THE NUMBER OF RULES USED FOR EACH PROTOCOL VERSION.

Subject	Protocol	#R	Sub.	Conf.	FP	Pend.
OpenSSL	TLS 1.3	312	14	9	1	4
wolfSSL	TLS 1.3	312	23	15	1	7
Mbed TLS	TLS 1.3	312	17	11	1	5
openHiTLS	TLS 1.3	312	19	12	1	6
Mosquitto	MQTT 5.0	164	21	13	1	7
Mosquitto	MQTT 3.1.1	127	16	10	0	6
wolfMQTT	MQTT 3.1.1	127	22	14	1	7
quiche	QUIC	286	24	15	1	8
ngtcp2	QUIC	286	18	11	1	6
libcoap	CoAP	143	18	11	1	6
BoringSSL	DTLS 1.3	198	12	5	0	7
Total	–	1230	204	126	9	69

In total, SpecField submitted 204 conformance-deviation reports. Maintainers have confirmed or fixed 126 of them, rejected 9 as false positives, and 69 remain pending. Two confirmed reports correspond to previously unknown vulnerabilities and were assigned CVE identifiers after disclosure.

Our precision denominator is the adjudicated subset, not all submitted reports: $\mathrm{P}_{adj} = \mathrm{Confirmed}/(\mathrm{Confirmed} + \mathrm{FP})$. To evaluate the bug-finding capability of different model families, Table VI compare their report outcomes under the same pipeline setting. We fix all non-model components to ensure a fair comparison across models. Specifically, each model uses the same extracted rules, code-retrieval budget, validators, adjudication protocol, and test-agent settings. The code-retrieval budget includes the number of candidate files, candidate variables, and code snippets that can be retrieved for each rule, as well as the maximum number of evidence-completion rounds. Therefore, different models are given rule sets and code evidence of the same scale, and the differences mainly come from the models' semantic understanding, code reasoning, and report-generation capabilities. Prompt differences are limited to model-specific formatting constraints needed to obtain schema-valid JSON outputs.

The results show that GPT-5.4 provides the most accurate reports, achieving the highest adjusted precision across the evaluated protocol versions. DeepSeek finds fewer confirmed reports in total than GPT-5.4, but contributes more unique confirmed reports, indicating that model diversity can expose additional deviations beyond those found by the strongest single model. These results suggest that stronger models are useful for high-precision reporting, while running multiple model families can increase the diversity of discovered implementation deviations. Provider-level cost differences are

TABLE VI
PER-MODEL COMPARISON OF CONFORMANCE-DEVIATION DETECTION. UNIQ. DENOTES CONFIRMED REPORTS FOUND ONLY BY THAT MODEL FAMILY. ADJ.P EXCLUDES PENDING REPORTS FROM THE DENOMINATOR.

Protocol	#R	GPT-5.4					DeepSeek					Claude					Qwen				
		Ext.	Conf.	FP	Uniq.	Adj.P	Ext.	Conf.	FP	Uniq.	Adj.P	Ext.	Conf.	FP	Uniq.	Adj.P	Ext.	Conf.	FP	Uniq.	Adj.P
TLS 1.3	312	73	47	4	0	0.92	62	37	7	1	0.84	68	42	6	0	0.88	56	31	7	0	0.82
QUIC	286	42	26	2	0	0.93	36	21	4	1	0.84	39	24	4	1	0.86	33	18	4	0	0.82
MQTT 5.0	164	21	13	1	0	0.93	18	10	2	4	0.83	19	12	2	0	0.86	17	9	2	0	0.82
MQTT 3.1.1	127	38	24	1	1	0.96	32	19	3	0	0.86	34	21	3	2	0.88	28	16	4	0	0.80
CoAP	143	18	11	1	0	0.92	16	10	1	2	0.91	17	10	2	1	0.83	15	9	2	0	0.82
DTLS 1.3	198	12	5	0	0	1.00	12	5	1	0	0.83	11	5	2	1	0.71	10	4	2	1	0.67
Total	1230	204	126	9	1	-	176	102	18	8	-	188	114	19	5	-	159	87	21	1	-

analyzed separately in RQ4.

F. RQ4: Cost Analysis

To evaluate the model cost of SpecField, we use GPT-5.4 pricing as the cost model. Table VII breaks down the average cost per report by pipeline stage, including rule extraction, code-context extraction, consistency reasoning, and the verification-test agent. The largest cost comes from consistency reasoning, which costs $0.431 per report on average and accounts for 38.4% of the total cost. This is because a suspicious finding usually cannot be judged in a single round. Instead, it often requires multiple evidence-completion rounds, such as retrieving callers, macro definitions, error-propagation paths, or configuration branches. The verification-test agent is the second-largest cost component, accounting for 34.3%, mainly due to test-plan generation, harness adaptation, log interpretation, and reproduction-report preparation. Rule extraction accounts for 15.7%, because the rule set for the same protocol version can be reused across multiple implementations, amortizing its cost. Code-context extraction accounts for only 11.6%, mainly for filtering candidate files, implementation variables, and relevant code snippets before reasoning.

Figure 2 further compares the same stage-wise cost breakdown across the four model families used in Table VI. Claude has the highest estimated total cost, followed by GPT-5.4, while DeepSeek and Qwen are about 35% cheaper than GPT-5.4. These results show a trade-off between model cost and bug-finding capability: lower-cost models reduce analysis expense, but, as shown above, they generally produce fewer maintainer-confirmed fixes and more false positives. In practice, one possible deployment strategy is to use lower-cost models for initial screening and then use stronger models to recheck high-risk rules or suspicious reports.

G. RQ5: Ablation Study

Taking GPT-5.4 as the example, Table VIII reports a component buildup on the adjudicated report set. Direct Agent and Generic RAG are the two non-field-aware baselines, covering 70.1% and 64.7% of the reports, respectively. Field Rules adds field-rule representation, field-aware localization, and a single reasoning pass, increasing coverage to 78.9% while maintaining an adjusted precision of 0.87.

Iter. Reasoning adds the evidence-completion loop used to request missing callers, macro definitions, validation helpers, and error-propagation paths before a candidate is submitted, increasing coverage to 93.1%. Forced Binary is a diagnostic control: collapsing low-evidence cases into binary decisions increases false positives to 63, showing the risk of removing the explicit INSUFFICIENT state. Full System further adds the validation-test agent, reaching the highest coverage of 95.6% while maintaining an adjusted precision of 0.87. Overall, these results show that the field-centered design is the main driver of report discovery, while iterative evidence completion expands reachable evidence and validation testing mainly reduces false positives before final reporting.

TABLE VII
AVERAGE GPT-5.4 MODEL COST PER SUBMITTED REPORT.

Stage	Cost / Report	Share
Rule extraction	$0.176	15.7%
Code-context extraction	$0.130	11.6%
Consistency reasoning	$0.431	38.4%
Verification-test agent	$0.385	34.3%
Total	$1.122	100.0%

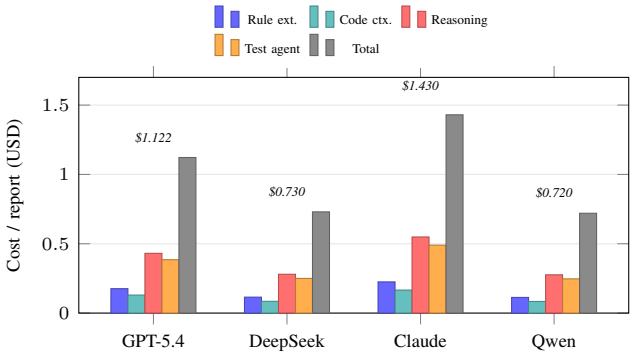

Fig. 2. Average model cost per submitted report by model family and pipeline stage. The GPT-5.4 bars correspond to Table VII

TABLE VIII
COMPONENT-BUILDUP RESULTS ON THE ADJUDICATED REPORT SET. COV. DENOTES THE PERCENTAGE OF THE 204 SUBMITTED REPORTS DISCOVERED BY EACH CONFIGURATION, NOT WHOLE-CORPUS RECALL.

Configuration	Sub.	Conf.	FP	Pend.	Adj.P	Cov. (%)
Direct Agent	143	72	24	47	0.75	70.1
Generic RAG	132	73	9	50	0.89	64.7
Field Rules	161	101	15	60	0.87	78.9
Iter. Reasoning	190	126	33	64	0.79	93.1
Forced Binary	175	111	63	64	0.64	85.8
Full System	195	126	19	69	0.87	95.6

H. Case Studies

Case 1: mbedTLS certificate-chain signature algorithms. RFC 8446 requires a TLS 1.3 server, when able, to provide a non-trust-anchor certificate chain signed with algorithms advertised by the client. SpecField extracts this requirement as a chain-level field rule and aligns it to mbedTLS's server certificate-selection logic. Static reasoning finds that mbedTLS uses the client's `signature_algorithms` to select the leaf certificate and `CertificateVerify` algorithm, but the evidence does not show a full-chain check over intermediate certificates. A validation test with a client advertising only `ecdsa_secp256r1_sha256` observes that mbedTLS accepts a server chain containing RSA-signed certificates. SpecField therefore reports a partial enforcement gap: leaf-level selection is enforced, but full-chain certificate-signature checking is missing. The issue was disclosed after confirmation and assigned a CVE.

Case 2: wolfSSL weak certificate-signature policy. SpecField identifies a weak-signature policy gap in wolfSSL certificate validation. RFC 8446 requires endpoints to abort when MD5 certificate signatures must be validated and recommends rejecting SHA-1-signed certificate chains, with limited exceptions for self-signed or trust-anchor certificates. In TLS 1.2 and TLS 1.3 runtime tests, wolfSSL rejects MD5-signed certificates when MD5 support is disabled, but accepts them under a more permissive build; SHA-1-signed self-signed certificates are accepted in all tested configurations. These results show that wolfSSL does not consistently block weak-signature certificates by default, exposing a secure-default and weak signature-acceptance risk. The issue was disclosed after confirmation and assigned a CVE.

I. Failure Analysis

The rejected reports mainly stem from two types of issues: compatibility interpretation and code alignment. Some reports involved compatibility behavior intentionally preserved by maintainers and were therefore not considered defects. Others were caused by ambiguous specification descriptions, where the mapping from fields to implementation variables or code locations became inaccurate, leading the system to analyze the wrong implementation context and preventing the dynamic test from truly reaching the target path. This suggests that more precise specification interpretation and more reliable code alignment are still needed when handling field rules with compatibility exceptions or unclear semantic boundaries.

V. DISCUSSION AND LIMITATIONS

Checking scope. SpecField primarily targets requirements that can be mapped to protocol fields, trigger conditions, and implementation behavior. It is well suited for checking local conformance deviations in parsing, validation, negotiation, and error handling. For global properties such as cross-session behavior or long-term state evolution, SpecField can only provide limited approximations and cannot offer completeness or correctness guarantees; strict conclusions still require separate modeling and verification.

Natural-language ambiguity. Ambiguity in standard text can affect rule extraction and conformance judgment. For example, a requirement such as "reject malformed messages" may not clearly specify the relevant field combinations, compatibility exceptions, or error-handling paths. In such cases, SpecField may misclassify reasonable compatibility behavior, or it may miss real defects. Therefore, for rules with unclear semantic boundaries or rules that depend on implementation context, the final conclusion still requires expert interpretation and maintainer feedback. Our artifact preserves the source text, reasoning evidence, and uncertainty notes to support subsequent review.

LLM dependence. SpecField uses LLMs for extraction, ranking, semantic filtering, reasoning, and test planning. The framework constrains these calls with schemas, field spaces, syntax evidence, and deterministic validators, but it does not eliminate model error. We mitigate this risk by retaining uncertainty, using low-temperature decoding, validating outputs, and requiring executable evidence for confirmed reports.

Source availability. The current prototype relies on source code to locate variables, call relationships, and validation logic, and therefore does not support closed-source implementations. For implementations produced by code generators, or implementations that encapsulate substantial parsing, validation, and error-handling logic in macros, semantic cues in the source code may be unstable or incomplete, which can affect the quality of specification-to-implementation alignment.

Language front-end. The current prototype mainly supports C/C++ protocol implementations. It combines syntactic analysis with LLM-assisted filtering to extract code context related to field accesses, conditional checks, and error handling from source code. For other programming languages, the system still requires corresponding language front-ends that provide equivalent code-analysis capabilities. In the future, we plan to extend this capability by integrating front-end analysis components for additional languages.

Result adjudication. Protocol conformance judgments cannot always be directly determined by the authors, because standards may be ambiguous and implementations may adopt different behaviors for compatibility. Therefore, system outputs should not be treated as final conclusions by themselves. We adjudicate reports as valid issues, false positives, or pending

based on maintainer feedback, independent review, and reproducible tests. This also means that some results depend on external feedback and follow-up validation, and the current statistics only reflect the adjudication status at the time of evaluation.

Cost and Human Effort. SpecField reduces model cost by caching standard-extraction results, compressing code contexts, and invoking the validation stage only for suspicious samples or samples with insufficient evidence. Nevertheless, multi-round reasoning and validation still incur higher model-calling costs than one-shot prompting, and report confirmation still requires some manual involvement. The manual effort mainly comes from reviewing candidate inconsistencies, including checking report accuracy, filtering a small number of false positives, confirming the execution results, and submitting reports to maintainers with follow-up communication.

Threats to validity. Our evaluation may be affected by manual adjudication and model stability. First, manually labeled ground truth may be influenced by reviewers' interpretations of ambiguous standards. Second, LLM nondeterminism may change extraction or reasoning results across runs. We mitigate these threats by using two-author adjudication, preserving raw evidence, separating pending reports from confirmed fixes, and measuring variance across repeated GPT-5.4 runs and different model families.

VI. Related Work

Specification-guided protocol analysis. Recent protocol-analysis systems increasingly use standard documents to guide implementation checking. RIBDetector extracts RFC-derived rules to identify implementation inconsistencies [10]; ParCleanse uses LLMs to translate RFC documents into protocol message specifications that serve as traceable quasi-oracles for parser validation [11]; RFCAudit builds semantic indexes over protocol code and uses an LLM agent to retrieve implementation context for RFC-related defect detection [12]; and ProtocolGuard combines LLM-guided program slicing with assertion-guided fuzzing to detect protocol non-compliance [13]. Other recent systems have similar goals: iPanda uses retrieval, LLM agents, and execution feedback to generate conformance tests [25], and AUTOSPEC synthesizes formal protocol specifications from natural-language documents and implementation feedback to support test generation [26]. These systems also use protocol standards to guide implementation analysis, but their outputs are usually rule-matching results, parser oracles, retrieved code snippets, or dynamic counterexamples. In contrast, SpecField models each check target as a field-level conformance claim, rather than remaining at the level of standard sentences or code fragments. Each claim binds the standard source, field semantics, applicability condition, implementation evidence, and evidence-sufficiency state, forming an evidence chain from specification fields to implementation variables and runtime behavior. This field-level modeling allows SpecField to characterize the correspondence between standard requirements and implementation behavior at a finer granularity, especially for subtle deviations in field values, state-dependent conditions, compound constraints, and error handling. Our evaluation shows that SpecField uncovers several implementation deviations that, to the best of our knowledge, had not been reported by prior standard-to-code checking work.

Protocol testing. Traditional conformance suites and protocol fuzzers provide strong executable evidence, but their oracles are usually hand-written, inferred from observed behavior, or tied to protocol-specific testing frameworks. TLS-Attacker and TLS-Anvil generate targeted TLS tests [27], [28]; protocol state fuzzing and automata-based analyses expose deviations in TLS/DTLS state machines [29]–[32]; and monitor-based testing encodes protocol requirements as external monitors and uses symbolic execution to find DTLS and QUIC deviations [33]. QUIC conformance studies also show that independent implementations can drift, regress, or diverge as specifications evolve [34]. Certificate-validation studies use adversarial, differential, symbolic, and RFC-guided inputs to test SSL/TLS implementations [35]–[39]. General protocol fuzzers such as AFLNet and StateAFL improve exploration of stateful network protocols [40]–[43]. These methods are effective at exploring protocol implementations and finding defects, especially when test targets, input-generation strategies, or oracles are already available. SpecField is complementary to them: traditional testing and fuzzing emphasize behavior triggering and defect discovery, whereas SpecField starts from natural-language standards, aligns field-level requirements with implementation code, and determines whether the implementation deviates from the standard.

Formal verification. Formal methods have been widely used for protocol security analysis and implementation verification. For example, prior TLS 1.3 work uses ProVerif and CryptoVerif to model core protocol logic and verify security properties [44]–[46]. Such methods can provide strong guarantees, but they typically rely on precisely defined abstract models or reference specifications, rather than directly addressing large-scale conformance checking between natural-language standards and existing code implementations.

Other work more directly checks conformance between protocol standards and implementations. Continuous formal verification of Amazon s2n uses Cryptol and SAW to prove equivalence between a TLS implementation and high-level specifications, and integrates verification into the development process [47]–[49]. Work on the OpenSSL TLS 1.3 handshake state machine constructs Cryptol models from both the RFC and the OpenSSL implementation, and compares state-transition sequences to check equivalence between the standard and the implementation [50]. SYMNV uses symbolic execution to generate test inputs and checks whether network daemons violate rules extracted from protocol specifications [51]. EverParse and follow-up work generate verified parsers and serializers from data-format specifications [52], [53]. Recent work has also explored end-to-end verification from high-level protocol models to production code [54], [55]. These efforts demonstrate the value of formal methods for protocol implementation verification, but they often require

substantial manual effort to construct and maintain models, specification languages, or proof scripts. When standards or implementations evolve, the corresponding formal artifacts must also be updated. SpecField does not aim to prove the correctness of an entire protocol or implementation. Instead, it identifies potential inconsistencies between natural-language standards and implementation code, and provides traceable evidence for subsequent confirmation.

VII. Conclusion

This paper studied how to identify field-level conformance deviations in source code implementations from natural-language protocol standards. We presented SpecField, which uses protocol fields as the linking points between standard text and implementation code, extracts structured rules from standards, locates relevant code evidence, and produces auditable conformance judgments. SpecField requires each judgment to be traceable to concrete standard spans, code contexts, and validation results, thereby reducing the risk of misclassification. Across multiple real protocol implementations, we submitted 204 reports, of which 126 were confirmed or fixed by maintainers, and discovered two previously unknown CVEs. These results show that starting from field requirements and combining them with code evidence is an effective way to uncover standard deviations in protocol implementations.

Data Availability. Our evaluation is designed to support reviewer inspection without violating double-blind review. For reports that have been confirmed and fixed by maintainers, the artifact uses blinded identifiers and provides the normative standard span, implementation evidence, inconsistency explanation, and reproduction scripts. For reports that are still pending or may involve undisclosed risks, we do not provide full reproduction materials in order to avoid amplifying potential security impact. All code, scripts, and blinded reports are included in an anonymous GitHub artifact repository submitted with the paper: https://anonymous.4open.science/r/SpecField-Artifact.

References

[1] T. Dierks and E. Rescorla, "The transport layer security (TLS) protocol version 1.2," RFC 5246, 2008, https://www.rfc-editor.org/rfc/rfc5246.

[2] E. Rescorla, "The transport layer security (TLS) protocol version 1.3," RFC 8446, 2018, https://www.rfc-editor.org/rfc/rfc8446.

[3] OASIS, "MQTT version 3.1.1," 2014, https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html.

[4] ——, "MQTT version 5.0," 2019, https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

[5] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, 2021, https://www.rfc-editor.org/rfc/rfc9000.

[6] M. Thomson and S. Turner, "Using TLS to secure QUIC," RFC 9001, 2021, https://www.rfc-editor.org/rfc/rfc9001.

[7] J. Iyengar and I. Swett, "QUIC loss detection and congestion control," RFC 9002, 2021, https://www.rfc-editor.org/rfc/rfc9002.

[8] S. Bradner, "Key words for use in RFCs to indicate requirement levels," RFC 2119, 1997, https://www.rfc-editor.org/rfc/rfc2119.

[9] B. Leiba, "Ambiguity of uppercase vs lowercase in RFC 2119 key words," RFC 8174, 2017, https://www.rfc-editor.org/rfc/rfc8174.

[10] J. Chen, F. Li, M. Xu, J. Zhou, and W. Huo, "RIBDetector: An RFC-guided inconsistency bug detecting approach for protocol implementations," in *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 2022, pp. 641–651.

[11] M. Zheng, D. Xie, Q. Shi, C. Wang, and X. Zhang, "Validating network protocol parsers with traceable RFC document interpretation," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1772–1794, 2025.

[12] M. Zheng, C. Wang, X. Liu, J. Guo, S. Feng, and X. Zhang, "RFCAudit: AI agent for auditing protocol implementations against RFC specifications," in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*, 2025, pp. 1221–1233.

[13] X. Song, L. Pei, J. Wu, Y. Zeng, G. He, C. Zuo, X. Liu, Q. Zhao, and S. Guo, "ProtocolGuard: Detecting protocol non-compliance bugs via LLM-guided static analysis and dynamic verification," in *Proceedings of the Network and Distributed System Security Symposium*. The Internet Society, 2026.

[14] M. Brunsfeld, "Tree-sitter: An incremental parsing system for programming tools," 2018, https://tree-sitter.github.io/tree-sitter/.

[15] OpenSSL Project, "OpenSSL's handling of HelloRetryRequest with an empty cookie is non-compliant with RFC 8446: The handshake is not terminated immediately," 2026, gitHub issue #30868. https://github.com/openssl/openssl/issues/30868.

[16] ——, "OpenSSL: Cryptography and SSL/TLS toolkit," 2026, https://www.openssl.org/.

[17] wolfSSL Inc., "wolfSSL embedded SSL/TLS library," 2026, https://www.wolfssl.com/.

[18] Mbed TLS Project, "Mbed TLS," 2026, https://www.trustedfirmware.org/projects/mbed-tls/.

[19] Google, "BoringSSL," 2026, https://boringssl.googlesource.com/boringssl/.

[20] Cloudflare, "quiche: Savoury implementation of the QUIC transport protocol and HTTP/3," 2026, https://github.com/cloudflare/quiche.

[21] T. Tsujikawa, "ngtcp2: An implementation of QUIC," 2026, https://github.com/ngtcp2/ngtcp2.

[22] Eclipse Foundation, "Eclipse Mosquitto: An open source MQTT broker," 2026, https://mosquitto.org/.

[23] wolfSSL Inc., "wolfMQTT client library," 2026, https://github.com/wolfSSL/wolfMQTT.

[24] libcoap Project, "libcoap: A CoAP implementation," 2026, https://libcoap.net/.

[25] X. Sun, F. Dang, S. Jiang, J. Xu, K. Liu, X. Miao, Z. Yang, W. Zhang, H. Lu, Y. Zheng, and Y. Liu, "iPanda: An LLM-based agent for automated conformance testing of communication protocols," 2025, arXiv:2507.00378.

[26] K. Liu, D. Chakraborty, A. Liggesmeyer, and A. Zeller, "AUTOSPEC: Synthesizing precise protocol specs from natural language for effective test generation," 2025, arXiv:2511.17977.

[27] J. Somorovsky, "TLS-Attacker: A framework for testing TLS libraries," in *Proceedings of the 10th USENIX Workshop on Offensive Technologies*, 2016.

[28] M. Maehren, P. Nieting, S. Hebrok, R. Merget, J. Somorovsky, and J. Schwenk, "TLS-Anvil: Adapting combinatorial testing for TLS libraries," in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 215–232.

[29] J. de Ruiter and E. Poll, "Protocol state fuzzing of TLS implementations," in *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, 2015, pp. 193–206.

[30] P. Fiterau-Brostean, B. Jonsson, R. Merget, J. de Ruiter, K. Sagonas, and J. Somorovsky, "Analysis of DTLS implementations using protocol state fuzzing," in *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 2020, pp. 2523–2540.

[31] P. Fiterau-Brostean, B. Jonsson, K. Sagonas, and F. Tåquist, "Automata-based automated detection of state machine bugs in protocol implementations," in *30th Annual Network and Distributed System Security Symposium (NDSS 2023)*. The Internet Society, 2023.

[32] B. Beurdouche, K. Bhargavan, A. Delignat-Lavaud, C. Fournet, M. Kohlweiss, A. Pironti, P.-Y. Strub, and J. K. Zinzindohoue, "A messy state of the union: Taming the composite state machines of TLS," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2015.

[33] H. Asadian, P. Fiterau-Brostean, J. de Ruiter, and E. Poll, "Monitor-based testing of network protocol implementations," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation*. IEEE, 2024, pp. 403–414.

[34] M. Piraux and O. Bonaventure, "Observing the evolution of QUIC implementations," in *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC*, 2018, pp. 8–14.

[35] C. Brubaker, S. Jana, B. Ray, S. Khurshid, and V. Shmatikov, "Using Frankencerts for automated adversarial testing of certificate validation in SSL/TLS implementations," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2014.

[36] Y. T. Chen and Z. Su, "Guided differential testing of certificate validation in SSL/TLS implementations," in *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 793–804.

[37] C. Chen, C. Tian, Z. Duan, and L. Zhao, "RFC-directed differential testing of certificate validation in SSL/TLS implementations," in *Proceedings of the 40th International Conference on Software Engineering*, 2018, pp. 859–870.

[38] Y. Chen, H. Su, Y. He, S. Jana, S. Chaudhuri, and X. Wang, "HVLearn: Automated black-box analysis of hostname verification in SSL/TLS implementations," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2017.

[39] S. Y. Chau, O. Chowdhury, E. Hoque, H. Ge, A. Kate, C. Nita-Rotaru, and N. Li, "SymCerts: Practical symbolic execution for exposing noncompliance in X.509 certificate validation implementations," in *Proceedings of the IEEE Symposium on Security and Privacy*, 2017, pp. 503–520.

[40] V.-T. Pham, M. Böhme, and A. Roychoudhury, "AFLNet: A greybox fuzzer for network protocols," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation*, 2020, pp. 460–465.

[41] R. Natella, "StateAFL: Greybox fuzzing for stateful network servers," *Empirical Software Engineering*, vol. 27, no. 7, p. 191, 2022.

[42] R. Natella and V.-T. Pham, "ProFuzzBench: A benchmark for stateful protocol fuzzing," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 662–665.

[43] J. Ba, M. Böhme, Z. Mirzamomen, and A. Roychoudhury, "Stateful greybox fuzzing," in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 3255–3272.

[44] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," *Foundations and Trends in Privacy and Security*, vol. 1, no. 1–2, pp. 1–135, 2016.

[45] ——, "A computationally sound mechanized prover for security protocols," *IEEE Transactions on Dependable and Secure Computing*, vol. 5, no. 4, pp. 193–207, 2008.

[46] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe, "A comprehensive symbolic analysis of TLS 1.3," in *Proceedings of the ACM Conference on Computer and Communications Security*, 2017.

[47] J. R. Lewis and B. Martin, "Cryptol: High assurance, retargetable crypto development and validation," Galois Connections white paper, 2003. [Online]. Available: https://cryptol.net/files/cryptol_whitepaper.pdf

[48] R. Dockins, A. Foltzer, J. Hendrix, B. Huffman, D. McNamee, and A. Tomb, "Constructing semantic models of programs with the Software Analysis Workbench," in *Proceedings of the International Conference on Verified Software: Theories, Tools, and Experiments*, 2016.

[49] A. Chudnov, N. Collins, B. Cook, J. Dodds, B. Huffman, C. MacCárthaigh, S. Magill, E. Mertens, E. Mullen, S. Tasiran, A. Tomb, and E. M. Westbrook, "Continuous formal verification of amazon s2n," in *Proceedings of the International Conference on Computer Aided Verification*, 2018, pp. 430–446.

[50] J. Guan, H. Li, X. D. Li, X. L. Wang, B. Wang, Q. Wang, S. Qin, M. He, M. Armanuzzaman, and Z. Zhao, "Formally verifying the state machine of TLS 1.3 handshake in OpenSSL," in *Proceedings of the IEEE Conference on Computer Communications*, 2025.

[51] J. Song, T. Ma, C. Cadar, and P. Pietzuch, "Rule-based verification of network protocol implementations using symbolic execution," in *Proceedings of the International Conference on Computer Communications and Networks*, 2011, pp. 1–8.

[52] T. Ramananandro, A. Delignat-Lavaud, C. Fournet, N. Swamy, T. Chajed, N. Kobeissi, and J. Protzenko, "EverParse: Verified secure Zero-Copy parsers for authenticated message formats," in *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, 2019, pp. 1465–1482.

[53] N. Swamy, T. Ramananandro, A. Rastogi, I. Spiridonova, H. Ni, D. Malloy, J. Vazquez, M. Tang, O. Cardona, and A. Gupta, "Hardening attack surfaces with formally proven binary format parsers," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2022, pp. 31–45.

[54] J. C. Pereira, T. Klenze, S. Giampietro, M. Limbeck, D. Spiliopoulos, F. A. Wolf, M. Eilers, C. Sprenger, D. Basin, P. Müller, and A. Perrig, "Protocols to code: Formal verification of a next-generation internet router," 2024, arXiv:2405.06074.

[55] L. Arquint, M. Schwerhoff, V. Mehta, and P. Müller, "A generic methodology for the modular verification of security protocol implementations," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2023, extended version: arXiv:2212.02626.